

Task 4: Build Your First Machine Learning Model

Dataset Used: Iris Dataset

Model Used: Decision Tree Classifier

Learning Type: Supervised Learning

Problem Type: Classification

1. Introduction

In this task, a basic Machine Learning classification model was developed using the **Iris dataset** and the **Scikit-learn** library.

The objective is to train a Machine Learning model that can predict the species of an Iris flower based on its sepal and petal measurements.

The three possible flower species in the dataset are:

- Iris-setosa
- Iris-versicolor
- Iris-virginica

A **Decision Tree Classifier** was selected because it is a simple and easy-to-understand classification algorithm.

2. Loading the Dataset

The Iris dataset was loaded using Pandas. The dataset contains **150 flower records and 6 columns**.

The four flower measurements are used as input features, while the Species column is used as the target variable.

The Id column is not used for prediction because it is only an identification number.

[1]:

```
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import LabelEncoder
from sklearn.tree import DecisionTreeClassifier
from sklearn.metrics import accuracy_score
```

[2]:

```
df = pd.read_csv("Iris.csv")

print("Dataset loaded successfully!")
print("Dataset shape:", df.shape)
```

Dataset loaded successfully!
Dataset shape: (150, 6)

3. Preparing X and y

The dataset was divided into two parts:

X — Features

The following four measurements were selected as features:

- SepalLengthCm
- SepalWidthCm
- PetalLengthCm
- PetalWidthCm

y — Target

The Species column was selected as the target variable.

The purpose of the model is to learn the relationship between the flower measurements in **X** and the species in **y**.

[3]:

```
features = [  
    "SepalLengthCm",  
    "SepalWidthCm",  
    "PetalLengthCm",  
    "PetalWidthCm"  
]  
  
X = df[features]  
y = df["Species"]  
  
print("Features (X):")  
print(X.head())  
  
print("\nTarget (y):")  
print(y.head())
```

Features (X):

	SepalLengthCm	SepalWidthCm	PetalLengthCm	PetalWidthCm
0	5.1	3.5	1.4	0.2
1	4.9	3.0	1.4	0.2
2	4.7	3.2	1.3	0.2
3	4.6	3.1	1.5	0.2
4	5.0	3.6	1.4	0.2

Target (y):

```
0    Iris-setosa  
1    Iris-setosa  
2    Iris-setosa  
3    Iris-setosa  
4    Iris-setosa
```

Name: Species, dtype: object

4. Encoding the Target Variable

The Species column contains categorical text values. Before training the model, these categories were converted into numerical labels using **LabelEncoder**.

The three species were identified as separate classes:

- Iris-setosa
- Iris-versicolor
- Iris-virginica

This conversion allows the classification algorithm to work with the target data in numerical form.

```
[4]:
label_encoder = LabelEncoder()

y_encoded = label_encoder.fit_transform(y)

print("Encoded target values:")
print(y_encoded[:10])

print("\nSpecies classes:")
print(label_encoder.classes_)

Encoded target values:
[0 0 0 0 0 0 0 0 0 0]

Species classes:
['Iris-setosa' 'Iris-versicolor' 'Iris-virginica']
```

5. Splitting the Dataset

The dataset was divided into training and testing data using an **80:20 ratio**.

- **Training data:** 120 records
- **Testing data:** 30 records

The training data is used to teach the Decision Tree model, while the testing data is kept separate to evaluate the model's performance on unseen data.

A `random_state` was used to make the split reproducible, and stratification was used to maintain a balanced representation of the different Iris species.

```
[5]:
X_train, X_test, y_train, y_test = train_test_split(
    X,
    y_encoded,
    test_size=0.2,
    random_state=42,
    stratify=y_encoded
)

print("Training data:", X_train.shape)
print("Testing data:", X_test.shape)

Training data: (120, 4)
Testing data: (30, 4)
```

6. Creating the Decision Tree Classifier

A **Decision Tree Classifier** was selected as the Machine Learning model.

A Decision Tree makes predictions by creating a series of decision rules based on the input features. It divides the data into smaller groups until it can make a classification decision.

For the Iris dataset, the model can learn patterns in measurements such as petal length and petal width to distinguish between different Iris species.

```
[6]:  
model = DecisionTreeClassifier(random_state=42)  
print("Decision Tree Classifier created successfully!")  
Decision Tree Classifier created successfully!
```

7. Training the Model

The Decision Tree model was trained using the training dataset.

During training, the model learns relationships between the four flower measurements and the corresponding species labels.

The model uses these learned patterns to make predictions when new flower measurements are provided.

```
[7]:  
model.fit(X_train, y_train)  
print("Model trained successfully!")  
Model trained successfully!
```

8. Making Predictions

After training, the model was given the testing data. The model used the patterns it learned during training to predict the species of the test flowers.

The predictions were stored and then compared with the actual species values.

```
[8]:  
y_pred = model.predict(X_test)  
print("Predictions:")  
print(y_pred)  
Predictions:  
[0 2 1 1 0 1 0 0 2 1 2 2 2 1 0 0 0 1 1 2 0 2 1 1 2 2 1 0 2 0]
```

9. Actual vs Predicted Results

The actual species and the species predicted by the Decision Tree model were displayed side by side.

This comparison helps us understand whether the model is correctly identifying the flower species.

For example, if the actual value is **Iris-setosa** and the predicted value is also **Iris-setosa**, the model has made a correct prediction for that record.

```
Actual = label_encoder.inverse_transform(y_test),  
"Predicted": label_encoder.inverse_transform(y_pred)  
})  
  
print(results.to_string(index=False))
```

Actual	Predicted
Iris-setosa	Iris-setosa
Iris-virginica	Iris-virginica
Iris-versicolor	Iris-versicolor
Iris-versicolor	Iris-versicolor
Iris-setosa	Iris-setosa
Iris-versicolor	Iris-versicolor
Iris-setosa	Iris-setosa
Iris-setosa	Iris-setosa
Iris-virginica	Iris-virginica
Iris-versicolor	Iris-versicolor
Iris-virginica	Iris-virginica
Iris-virginica	Iris-virginica
Iris-virginica	Iris-virginica
Iris-versicolor	Iris-versicolor
Iris-setosa	Iris-setosa
Iris-setosa	Iris-setosa
Iris-setosa	Iris-setosa
Iris-versicolor	Iris-versicolor
Iris-versicolor	Iris-versicolor
Iris-virginica	Iris-virginica
Iris-setosa	Iris-setosa
Iris-virginica	Iris-virginica
Iris-versicolor	Iris-versicolor
Iris-virginica	Iris-versicolor
Iris-virginica	Iris-virginica
Iris-versicolor	Iris-virginica
Iris-versicolor	Iris-versicolor
Iris-setosa	Iris-setosa
Iris-virginica	Iris-virginica
Iris-setosa	Iris-setosa

10. Model Accuracy

The accuracy of the Decision Tree model was calculated using the actual and predicted values.

Accuracy represents the proportion of correct predictions made by the model out of all the predictions.

The accuracy obtained from the notebook was used to evaluate the performance of the classification model.

```
[10]:
```

```
accuracy = accuracy_score(y_test, y_pred)

print("Model Accuracy:", accuracy)
print("Model Accuracy (%)ate:", accuracy * 100)
```

```
Model Accuracy: 0.9333333333333333
```

```
Model Accuracy (%): 93.33333333333333
```

11. How the Decision Tree Model Works

A Decision Tree works by making a series of decisions based on the input features.

For example, the model may learn that certain petal measurements are strongly associated with a particular Iris species. It creates decision rules based on these patterns and uses them to classify new observations.

The process can be summarized as:

Training Data → Learn Patterns → Build Decision Rules → Testing Data → Make Predictions

The model does not need to be explicitly told every rule. Instead, it learns useful decision rules from the training data.

12. Machine Learning Workflow

The complete workflow followed in this task was:

Iris Dataset

↓

Prepare Features (X) and Target (y)

↓

Encode Target

↓

Split Data into Training and Testing Sets

↓

Create Decision Tree Classifier

↓

Train the Model

↓

Make Predictions

↓

Compare Actual and Predicted Values



Calculate Accuracy

Conclusion

In this task, a basic Machine Learning classification model was successfully developed using the Iris dataset and a **Decision Tree Classifier** from Scikit-learn.

The dataset was prepared by selecting the appropriate features and target variable. The target species were encoded into numerical labels, and the dataset was divided into training and testing sets using an 80:20 ratio.

The Decision Tree model was trained using the training data and then used to predict the species of flowers in the testing data. The actual and predicted results were compared, and the model's accuracy was calculated.

This task demonstrates the basic Machine Learning workflow of **data preparation, model training, prediction, and evaluation**.

Final Takeaway

A Decision Tree Classifier learns patterns from training data and uses those learned decision rules to classify new, unseen data into the appropriate category. 🌱